

Comment je mets à jour mes 3 apps TaxiLink ?

Expliqué comme à un enfant de 10 ans — corriger un bug, ajouter une fonctionnalité, sortir une nouvelle version

Imagine que tes 3 apps TaxiLink sont sorties dans les magasins Apple et Google. **500 chauffeurs, 50 patrons et 2000 patients** les utilisent. Bravo ! 🎉

Mais lundi matin, un chauffeur t'appelle : "Le tarif CPAM s'affiche en double sur l'écran fin de course."

👉 Comment tu envoies la correction sur les **2 550 téléphones** qui utilisent ton appli, sans qu'ils aient à faire quoi que ce soit ? C'est ce qu'on va t'expliquer.

1. Les 2 sortes de mises à jour

Il y a deux façons très différentes de modifier ton appli une fois qu'elle est dans les stores. Tu vas en utiliser **les deux**, mais pas pour les mêmes raisons.



La mise à jour **MAGIQUE (OTA)**

L'appli se corrige toute seule en arrière-plan, en quelques minutes, **sans que personne télécharge rien**. Apple et Google ne sont même pas au courant.

Délai : instantané



La mise à jour **OFFICIELLE (native)**

Tu envoies une **nouvelle version complète** à Apple et Google. Ils la relisent, ils valident, puis les utilisateurs la voient apparaître dans le store et la téléchargent.

Délai : 1 à 3 jours

L'image facile à comprendre

⚡ **OTA**, c'est comme un livre **magique** : pendant que tu dors, les pages se réécrivent toutes seules avec les corrections du jour. Tu te lèves le matin, tu rouvres ton livre, c'est déjà à jour.

📦 **Native**, c'est comme rééditer le livre en imprimerie, l'envoyer dans toutes les librairies, et attendre que les gens viennent chercher la nouvelle édition. C'est plus long, mais c'est obligatoire pour les **grandes nouveautés**.

2. Quand utiliser laquelle ?

Si tu veux changer ça...	Tu utilises...	Pourquoi
Corriger un bug dans le calcul du tarif CPAM	⚡ OTA	C'est juste du code JavaScript, pas de pièce native
Changer un texte mal orthographié	⚡ OTA	Texte = pas natif
Ajouter un nouvel écran "statistiques de la semaine"	⚡ OTA	Tant qu'il n'utilise pas de nouveau plugin natif
Modifier les couleurs du dashboard	⚡ OTA	Style = pas natif
Ajouter le scan NFC de la carte Vitale	📦 Native	NFC est un plugin natif, il faut rebuild
Ajouter une nouvelle permission (genre Bluetooth)	📦 Native	Apple et Google doivent valider la permission
Mettre à jour Expo SDK 51 → 52	📦 Native	Code natif change, rebuild obligatoire
Changer l'icône de l'app ou le splash screen	📦 Native	Ces ressources sont compilées dans le binaire
Sortir une "version 2.0" avec gros changements	📦 Native	Toujours par sécurité + visibilité dans les stores

✓ La règle simple

Si tu modifies **uniquement** du code TypeScript, des images ou du texte → **OTA**. Si tu touches à **n'importe quel plugin natif**, à l'icône, aux permissions ou au SDK → **build natif**.

3. Tes outils, dans ton cas TaxiLink

Tu as déjà plein d'outils en place qui vont te servir pour les mises à jour. Voilà à quoi ils servent dans ce contexte.

EAS Update (~30 €/mois)

Inclus dans EAS Production

L'**usine magique des mises à jour OTA**. Tu lances eas update, et en 30 secondes tes 2 550 utilisateurs ont la nouvelle version au prochain démarrage de l'appli.

EAS Build

Inclus dans EAS Production

L'usine cloud qui fabrique les **fichiers .ipa (Apple) et .aab (Google)** à envoyer aux stores. ~15-30 min par app.

EAS Submit

Gratuit

Le **postier automatique** qui envoie les fichiers à Apple et Google sans que tu touches à Xcode ou Android Studio.

Sentry (déjà chez toi)

~0 € (mutualisé avec ton site)

Le **radar à bugs**. Si l'appli plante chez un chauffeur, Sentry te notifie en 30 secondes avec la ligne de code en cause. Indispensable pour réagir vite.

App Store Connect (Apple)

Gratuit (compte 99 €/an)

Le **tableau de bord Apple** où tu vois tes versions, les statistiques, les crashes. C'est là que tu cliques "Submit for review".

Google Play Console

Gratuit (compte 25 € à vie)

Le **tableau de bord Google**. Tu y gères les versions, les testeurs, le déploiement progressif.

4. Trois scénarios concrets dans ta vraie vie TaxiLink

Scénario 1 : un chauffeur trouve un bug grave dans le tarif CPAM Délai :

30 min

Lundi 8h30 — Karim, chauffeur à Marseille, t'appelle : *"Le total de la course s'affiche 2 fois sur l'écran fin de course depuis ce matin."*

- **8h35** — Tu ouvres Sentry. Tu vois 23 occurrences du même bug depuis 7h. Pas une coïncidence.
- **8h40** — Avec Claude Code, tu identifies le coupable : un calcul dupliqué dans `fareCalculator.ts`.
- **8h50** — Fix écrit, tests passent localement.
- **8h52** — Tu lances : `eas update --branch production --message "Fix double tarif CPAM"`
- **8h53** — L'usine Expo upload la mise à jour.
- **8h55-9h05** — Au fur et à mesure que les chauffeurs ouvrent l'appli, ils téléchargent silencieusement la correction.
- **9h10** — Tu rappelles Karim : *"Ferme et rouvre ton appli."* Il le fait. Le bug a disparu.
- **9h15** — Sentry est revenu à zéro nouvelle occurrence. 

Total : 30 minutes du signalement à la correction sur 500 téléphones. Apple et Google n'ont rien fait.

Scénario 2 : tu ajoutes le scan NFC de la carte Vitale (vraie nouvelle fonctionnalité)

Délai : 4 jours

Mardi matin — Tu décides de simplifier la vie des chauffeurs : au lieu de tout taper, ils approchent la carte Vitale du téléphone et hop, infos patient récupérées.

- **Mardi 10h** — Tu ajoutes le plugin `react-native-nfc-manager` dans `app.json`.
- **Mardi 10h-17h** — Avec Claude Code, tu codes l'écran de scan + le service de lecture NFC.
- **Mardi 18h** — Tu bumpes la version : `"version": "1.5.0", "buildNumber": "47", "versionCode": 47`.
- **Mardi 18h05** — Tu lances : `eas build --platform all --profile production`
- **Mardi 18h35** — Les 2 builds (iOS + Android) sont prêts.
- **Mardi 18h40** — Tu lances : `eas submit --platform all`
- **Mardi 18h45** — Apple et Google reçoivent les fichiers.
- **Mercredi 14h** — Google Play approuve. Tu lances un staged rollout à 5%.
- **Mercredi 16h** — Apple : *"Pourquoi cette permission NFC ? Donnez une vidéo de démo."*
- **Mercredi 17h** — Tu enregistres une vidéo de 30s + tu réponds.
- **Jeudi 11h** — Apple approuve. Tu lances un phased release iOS.
- **Vendredi-Lundi** — Sentry surveille. Pas de crash. Tu montes Google à 50% puis 100%.
- **Lundi suivant** — 100% des utilisateurs sur la 1.5.0 avec NFC actif. ✓

Total : 4 jours ouvrés dont 1 cycle de question Apple. Avec Claude Code, le code lui-même prend 1 journée.

Scénario 3 : Apple sort iOS 27 et casse une ancienne API de localisation

Délai : 1-2 semaines

Septembre — Apple sort iOS 27. Sentry te montre que **15 % des chauffeurs iPhone ont leur GPS qui freeze**.

- **J1** — Tu identifies que expo-location doit passer en version 17 → SDK Expo 52 requis.
- **J1-J3** — Tu mets à jour le SDK avec Claude Code, tu fixes les breaking changes (typiquement 5-10 endroits).
- **J4** — Tu testes sur ton iPhone perso + 2 testeurs beta.
- **J5** — Bump 1.6.0, build, submit.
- **J6-J8** — Apple review (sensible car GPS background, parfois 2 cycles).
- **J9-J14** — Phased release surveillé, et urgence sur les 15 % bloqués.

Total : 1-2 semaines. C'est le scénario le plus stressant car il y a une *deadline naturelle* imposée par Apple.

5. Le déploiement progressif (le plus important !)

L'image facile à comprendre

Imagine que tu inventes une nouvelle recette de gâteau pour ta classe. Tu ne vas pas en faire **30 d'un coup** et risquer que tout soit raté. Tu fais d'abord **un seul gâteau test**, tu le donnes à 1 ami, tu vois sa réaction. Si c'est bon, tu en fais 5. Si c'est encore bon, 15. Si tout le monde adore, tu fais les 30.

Avec une appli c'est pareil. On appelle ça le "**phased rollout**" ou "**déploiement progressif**".

Jour	iOS Apple (Phased Release)	Android Google (Staged Rollout)	Toi tu fais quoi ?
Jour 1	1 % des users	5 % des users (à toi de choisir)	Tu surveilles Sentry toutes les 30 min
Jour 2	2 %	10 % si Sentry clean	Surveillance Sentry
Jour 3	5 %	20 %	Tu lis les retours WhatsApp / email chauffeurs
Jour 4	10 %	50 %	Surveillance
Jour 5	20 %	100 %	Surveillance
Jour 6	50 %	—	Surveillance
Jour 7	100 %	—	🎉 Tout le monde est sur la nouvelle version

⚠ Et si Sentry montre un crash gros au jour 2 ?

- **Apple iOS** : tu cliques "Pause" dans App Store Connect → la diffusion s'arrête net (les 5 % qui ont déjà téléchargé restent dessus).
- **Google Play** : tu fais un **"halt rollout"** et tu peux même **revenir à la version précédente** ! C'est la fonction "rollback".
- **OTA** : tu peux republier en 30 secondes la version d'avant via `eas update --rollback-to-embedded`.

6. Les numéros de version (ne te trompe pas !)

L'image facile à comprendre

Imagine que ton appli est un livre. Le **numéro de version** (ex : 1.4.2) est ce qui s'affiche sur la couverture du livre, ce que tout le monde voit. Le **buildNumber / versionCode** (ex : 42) est un numéro *interne*, comme le numéro d'édition imprimé en bas du dos. Personne ne le voit, mais Apple et Google s'en servent pour ne pas se tromper.

Champ dans app.json	Exemple	Règle
"version"	"1.4.2"	Visible des utilisateurs. 3 chiffres : majeur.mineur.patch . Tu choisis.
"ios.buildNumber"	"42"	Interne Apple. Doit toujours augmenter de 1 à chaque submit.
"android.versionCode"	42	Interne Google. Doit toujours augmenter de 1 à chaque submit.

Convention pour TaxiLink

Type de changement	Numéro à bumper	Exemple
Fix de bug (OTA ou native)	Le dernier chiffre (patch)	1.4.2 → 1.4.3
Nouvelle fonctionnalité	Le 2 ^e chiffre (mineur)	1.4.3 → 1.5.0
Refonte majeure (genre nouveau design complet)	Le 1 ^{er} chiffre (majeur)	1.5.3 → 2.0.0

7. Les pièges spéciaux TaxiLink

Piège n°1 : la mise à jour OTA pendant que Apple review une autre version

Tu as soumis la 1.5.0 à Apple. Pendant que Apple regarde, tu pousses un OTA sur la 1.4.x en prod. **Apple voit la nouvelle UI sur le binaire 1.5.0 qu'il review** alors que ce binaire ne contient pas cette UI. Confusion garantie, possible rejet.

Solution : mettre les OTA en pause pendant la review (option dans EAS), ou aligner la branche OTA runtimeVersion avec le binaire.

Piège n°2 : casser le tarif CPAM en prod chauffeur

Le calcul tarif CPAM est **extra-sensible** chez toi : un mauvais arrondi = facturation patron qui s'effondre, ou pire, fraude involontaire à la Sécu. Pour cette zone du code, **jamais d'OTA direct**.

Règle : tout changement sur `fareCalculator.ts` ou `cpamRules.ts` = passe par un build natif + beta interne 48h avant prod.

Piège n°3 : oublier que les patients sont âgés

L'app patient sera utilisée par des seniors avec téléphones anciens (iPhone 8, Android Samsung de 2018). Une mise à jour qui marche chez toi peut **planter ou être lente** chez eux.

Solution : avoir un vieux iPhone et un vieux Android dans le stock test, et un staged rollout plus lent (10 jours pour le 1.0 → 100 %) sur l'app patient.

Piège n°4 : 3 apps = 3 calendriers de release

Tu auras 3 apps qui évoluent à des rythmes différents : la chauffeur change tous les jours, la patron une fois par mois, la patient rarement.

Solution : une **release notes** partagée par app, un canal Slack/Discord par app, et 3 colonnes "version live" dans un tableau dashboard.

Piège n°5 : le chauffeur qui n'ouvre pas l'appli pendant 3 semaines

Pour qu'un OTA prenne effet, il faut **ouvrir l'appli**. Si Karim part en vacances 3 semaines, il rouvre avec une version *très* en retard. Bug du jour ? Bug du jour 1 ? Bug du jour 5 ? Tout d'un coup.

Solution : écran "Mise à jour disponible, redémarrer maintenant" si l'écart de version est trop grand, et forcer un rebuild natif tous les ~3 mois pour absorber les OTA accumulés.

8. La cadence type de TaxiLink en croisière

Type d'action	Méthode	Fréquence	Qui fait ?
Fix de bug urgent	 OTA	Dès que Sentry alerte	Toi + Claude Code
Petite amélioration UX	 OTA	2 fois par semaine	Toi + Claude Code
Nouvelle fonctionnalité moyenne	 Native	1 fois par mois	Toi + Claude Code
Mise à jour Expo SDK	 Native	2 fois par an	Toi + Claude Code (1-2 jours)
Réponse à un nouveau iOS / Android	 Native	1 fois par an (sept-oct typiquement)	Toi + Claude Code
Refonte majeure (v2.0)	 Native	1 fois par an ou plus	Toi + Claude Code (gros chantier)

9. Le grand récap en une page

Ce qu'il faut retenir

- **OTA = magique, instantané.** Pour les bugs et les petites améliorations.
- **Native = officiel, 1 à 3 jours.** Pour les vraies nouveautés et tout ce qui touche au natif.
- **Toujours faire un déploiement progressif** (5 % → 100 %) pour éviter les catastrophes.

- **Sentry** est ton meilleur ami : il te prévient des bugs avant les chauffeurs.
- **Numéros de version** doivent toujours augmenter, sinon Apple et Google rejettent.
- **Le tarif CPAM** ne s'OTA jamais direct : toujours via build + beta interne 48h.



La promesse de vitesse

Avec ton stack TaxiLink (Expo + EAS + Sentry + Claude Code), tu peux :

- Corriger un bug sur 2 550 téléphones en **30 minutes**
- Déployer une nouvelle fonctionnalité moyenne en **4 jours**
- Réagir à un changement Apple/Google en **1-2 semaines**
- Sortir une refonte v2.0 en **4-6 semaines**

C'est **10 fois plus rapide** qu'une équipe classique sans IA il y a 5 ans.